

# Trading bot

Author: Yaroslav Riabikov

## Goals

Goal of this project is to build end-to-end automated trading bot with web analytics.

Main points:

1. Web crawler continuously scrapes news articles from <https://techcrunch.com/>.
2. Google Gemini API processes text and outputs instructions (Buy/Sell).
3. Bot processes instructions, edits portfolio, saves it and history of operations to database.
4. Webpage shows statistics collected throughout bot's run.

## Data

Data to make predictions was scraped from <https://techcrunch.com/> website I was using Latest News section. Bot read articles' links and then processed them one by one from oldest to newest. Processing articles includes reading them from webpage and sending to Google Gemini API to get response. To get stocks' prices I used yfinance library.

Database contains four tables:

1. bot\_variables – budget, last\_link (link of latest article that was processed)
2. stocks – stocks portfolio (name, count)
3. operations\_history – all information about each operation (stock, buy/sell, count, current stock price, remaining budget, time)
4. portfolio\_history – stores total portfolio wealth through the time

First two tables are used to store portfolio information, last two tables are used to provide statistics, evaluate bot's performance and track its behavior.

## Method description

### **0. Main.py reads database and creates Crawler, NewsAnalyzer and PortfolioOperator entities and starts Crawler's run**

Detailed description: main.py starts loop, in each operation it reads data from database and creates all bots. It calls process\_webpage method of Crawler class, which starts to read Latest News section in techcrunch.com. If problem occurs it will wait 1 minute and retry.

### **1. Crawler scrapes articles from techcrunch.com, sends to NewsAnalyzer**

Detailed description: Crawler using requests and BeautifulSoup reads Latest News section from techcrunch.com main page, gets links to certain articles that will be used in trading decisions. After getting all needed link we start processing loop, we reverse list of links to process them from oldest to newest article, which is logical order. In each iteration we read each article's page using requests and BeautifulSoup.

### **2. NewsAnalyzer sends data to Google Gemini API, gets response and returns output to Crawler**

Detailed description: after reading article's page and transforming it to text that is going to be consumed by LLM Crawler calls predict method of NewsAnalyzer class which sends article, current portfolio and available budget to LLM and asks for trading decision, after that output is transformed into python dictionary and returned to Crawler.

### **3. Crawler gets output from NewsAnalyser and sends to PortfolioOperator**

Detailed description: then if trading decision of LLM is not empty Crawler calls process\_data method of PortfolioOperator

#### 4. PortfolioOperator manages stocks, budget as NewsAnalyser advised

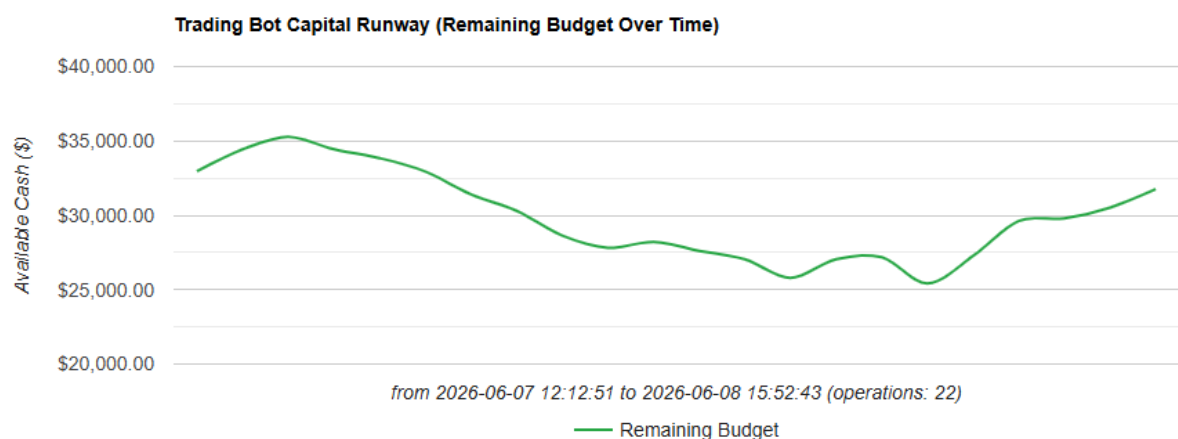
Detailed description: process\_data method initializes available budget per article, applies normalization to prevent LLM break budget or any other rules of safe trading. It reads dictionary with trading advices, gets each stock price using yfinance library and if price for such stock is listed, performs operations using sell\_stocks and buy\_stocks methods. After each operation is performed it saves information about operation to operations\_history table and also edits stocks and bot\_variables tables.

#### 5. Flask server with JS charts provides meaningful insights into bot's performance and behavior

Detailed description: server.py using SQL queries we collect all needed data from operations\_history and portfolio\_history tables. In main.html file we receive data and build charts. Basic structure was taken from developers.google.com and I added dynamic filling of charts and in first two line-charts I added specialized x-axis label which shows when first operation was performed and when last, since having all dates there was unreadable.

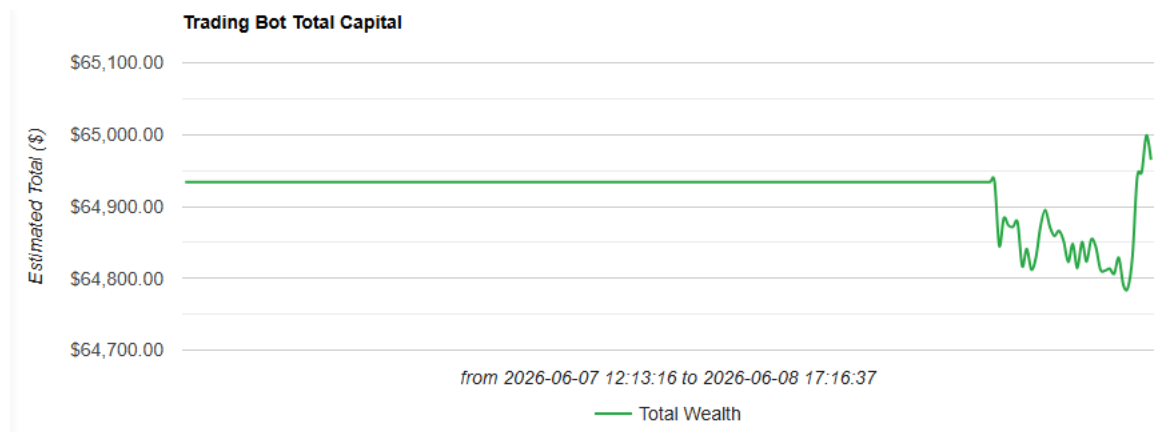
## Analysis

First chart is chart representing amount of money available to bot: y-axis represents available cash in dollars, x-axis represents timeline.



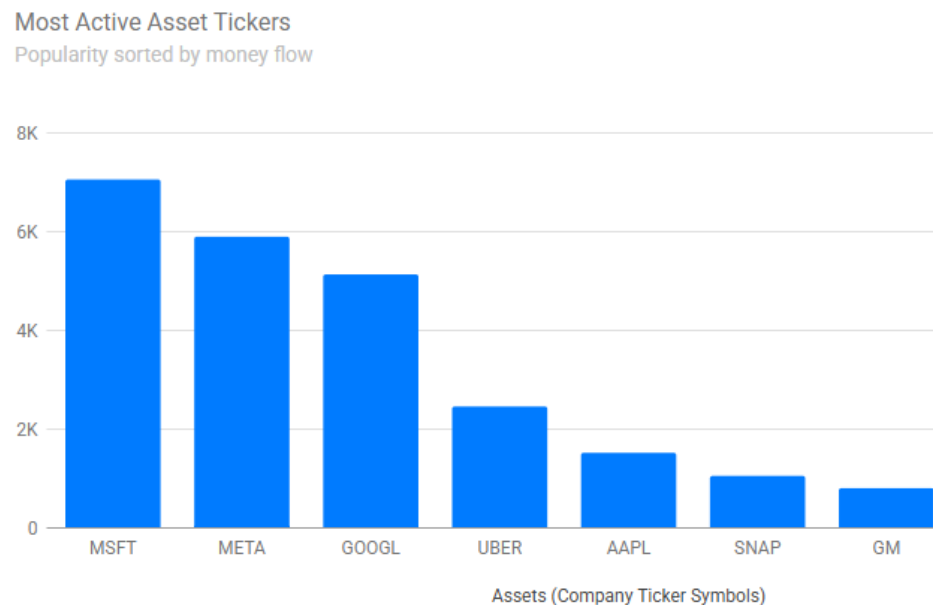
Here we can track if our bot always has enough money and how he spends cash he has. Because of limit per one article, it doesn't spend too much money in single run. We can see that bot doesn't struggle and has enough money. It's available resources variate around 30 thousand dollars, which is more than enough to trade mostly all known stocks actively and freely. We can see from label for x-axis that there were 22 operations in total and first one was on 2026-06-07 12:12:51 and last one was on 2026-06-08 15:52:43, which means that there was almost one operation per hour, which can be considered as pretty active trading. Nevertheless, if LLM was told to trade conservatively numbers of operations would have dropped.

Second chart is the most important for us: Total capital: y-axis represents estimated total in dollars, x-axis represents timeline.



We can see long flat line; it is all runs on weekends when stock market is closed. Then line becomes very messy, but it will have clearer results in long-term. This chart was supposed to be useful in long-term, it might seem not useful right now, but it is necessary if we are going to analyze system throughout longer period of time.

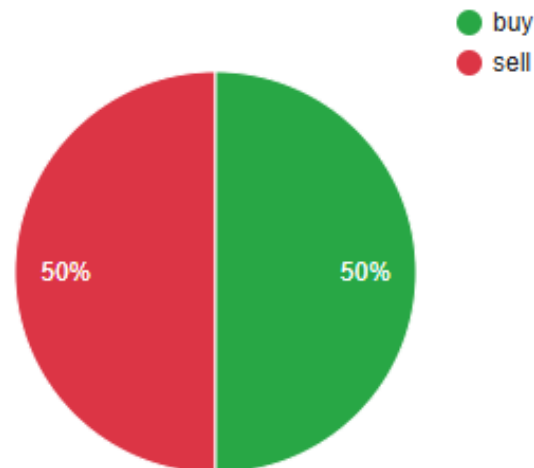
Then we have Barplot that shows us most active tickers: stocks, that bot trades the most: y-axis represents how much money did bot trade in terms of certain stock in dollars, x-axis represents stocks.



Here we can see such large companies as Microsoft, Meta, Google. It is expected, since we get a lot of news about known companies.

Last chart is pie chart: Buy/Sell operation friction:

**Trading Behavior Distribution (Total Volume Bought vs. Sold)**



It is really important for tracking bot's behavior. In my case bot perfectly balances between buying and selling. It means that we won't get such problems as not having money after some time trading or our bot only selling and not buying anything.

As a result, we can see that bot acts quite responsible, it can not spend all money in one go, it spends money gradually, he balances between buying and selling, which means that he will have available cash in long-term, if he will continue having same behavior. He trades mostly well-known stocks.

## Conclusion

The most difficult part of the project was to restrict LLM to output meaningless predictions. I needed to apply a lot of normalization so LLM will not break any rules and to ensure that it does not do anything that it was not supposed to do. Another important part was to make architecture clean and upgrade friendly. I decided to create Crawler as main bot and two more ones as supporters so no one will be responsible for too many tasks. However, for more complex systems it will be better for Crawler to be not the main entity, but supporter, since it will be responsible for way too many tasks. The easiest part was probably reading articles from the website. During the project I practiced myself in building architecture from scratch; discovered several interesting libraries; connected crawler, LLM, SQL, flask and JS charts.